

Brownfield Adoption Guide

Introduction

In the majority of the cases we start to work in a project that already has an existing repository. Either because the repository was inherited, or because the project has already started. The implication is clear: you won't be able to design the application from scratch with all the good software principles you know.

When you enter this new project, in my experience, most likely there are going to be gaps regarding the desired design of the system with respect to the current state of the system. Are these gaps easily solvable? If the answer is no, then you are probably facing a **legacy system**: a system that is basically difficult to modify.

Dealing with a legacy system is hard, but not impossible. In this guide I will explain how I successfully worked with a legacy system. Hopefully these lessons can work for you as well. My advice will complement the timeless book on this topic "Working Effectively with Legacy Code" by Michael Feathers.

What is a legacy system?

A legacy system is any piece of software that is difficult to change safely — not because it is old, but because it lacks the safety mechanisms that make change possible. The clearest signal: **lack of automated tests**. Without tests, you cannot know whether a change broke something until it breaks in production. This guide will help you build those mechanisms and use them to modernize a system incrementally.

Working Effectively with Legacy Code summary

1. **Name the problem precisely.** Before touching anything, write down exactly what behavior needs to change — vague goals lead to changes you can't verify.
2. **Write characterization tests.** Capture what the code *actually* does right now through a set of scenarios, not what it should do — this is your safety net before any change.
3. **Find a seam.** A seam is a point in the code where you can substitute different behavior without editing the code at that exact spot (e.g. passing in a dependency instead of creating it internally).
4. **Break the dependency at the seam.** Use the seam to isolate the piece you need to change from the rest of the system, so it can be tested on its own.

5. **Add the test you actually wanted.** Now that the code is isolated and testable, write the test for the new or changed behavior.
 6. **Make the smallest safe change.** Modify the code in the smallest increment that keeps every test — old and new — passing.
 7. **Refactor with confidence.** With characterization tests as a safety net, clean up the surrounding code without fear of silently breaking it.
-

Complementary suggestions to the book

Findings from Francisco Zenteno Smith (2026), "Why operations research practitioners resist software engineering practices and what changes their behavior," presented at the 2026 DSI Annual Conference, San Francisco, CA, USA.

In my experience, working with legacy code is more of a **Change Management** problem, rather than a **Software Problem**. Here I summarize the findings of the aforementioned paper. When I worked on this project, I based my actions on the change management book "Switch: How to Change Things When Change Is Hard" by Chip Heath and Dan Heath.

Make the problem visible

1. **Make the invisible visible.** Install a code-quality tool (like SonarQube) and a code coverage tool before you argue for change — people can't act on a cost they can't see.
2. **Keep feedback depersonalized.** Point at the tool's numbers, not at someone's code by name — singling a person out reads as an attack, not a lesson.
3. **Borrow authority from outside.** Show that the state-of-the-art practice is how skilled practitioners elsewhere already work, not your personal preference.

Make change the easy path

1. **Build the safety net before you ask for change.** A suite of characterization tests removes the single biggest reason people avoid refactoring: fear of silently breaking something.
2. **Change the default, not the willpower.** A pull request template and/or mandatory checklists in your CI pipeline make a missing test visible.
3. **Cut friction in a slow adjacent workflow.** Automating a painfully manual step buys you both time and credibility at once.
4. **Remove the specific technical barrier behind "that can't be tested."** Often the only real obstacle is that no one has shown it can be done — show how, then do it.

5. **Invest in software engineering training.** Usually the root cause of poor software is the current developers' practices. Assess if they need software engineering training, and if they do, make sure that happens sooner rather than later.

Make it theirs

1. **Model the practice yourself first.** Be the first to follow the template, write the test, or refactor the flagged code — credibility is key.
2. **Tie the practice to who they already are.** Frame it in the vocabulary of their own expertise, not as extra work bolted onto it.
3. **Use a real failure as the turning point.** A concrete emotional incident traceable to the old way of working shifts the conversation from "should we change" to "how fast can we."
4. **Give the change a public moment.** A presentation or recognition event where the team — not you — gets the credit is what makes the change outlive your involvement.

Know your terrain

1. **Check your position before you start.** A supportive manager will help a lot when investing in refactoring. Make sure you have some support before you start.
2. **Watch for shifts in who the team defers to.** A departure or new hire can change informal authority — recognize the opening rather than assuming only your effort moved things.
3. **Report quality and throughput together.** Showing that speed didn't drop is what defuses the belief that good practice and delivery pace trade off against each other. Make sure the key milestones of the project are being delivered on schedule.
4. **Expect old habits under pressure, not their disappearance.** The realistic goal is a changed default, not the permanent elimination of shortcuts.